

SmartPizza AI

Full-Stack Pizza Ordering & Delivery System
Capstone Project Documentation

Frontend: React | Backend: Spring Boot (Java) | Database: MySQL

1. Introduction

SmartPizza AI is a full-stack pizza ordering and delivery web application, similar in idea to food platforms like Swiggy or Zomato, but focused on pizza. It allows customers to browse a menu, add pizzas to a cart, apply coupons, pay online or by cash, track their order live, and download an invoice. The system supports three types of users (roles): Customer, Admin, and Delivery Partner.

The project is built in two parts: a React frontend (what the user sees) and a Spring Boot backend (the server and business logic), connected through REST APIs, with a MySQL database for storage.

2. Objectives

- Build a complete online pizza ordering system with three user roles.
- Provide secure login and role-based access using JWT tokens.
- Support cart, coupons, GST billing, and both online and cash-on-delivery payments.
- Provide live order tracking with an estimated delivery time (ETA).
- Give the admin a kitchen workflow, smart delivery assignment, and an analytics dashboard.
- Generate downloadable PDF invoices using real order data.

3. Technology Used

Layer	Technology	Purpose
Frontend	React, Vite, Tailwind CSS	User interface and screens
Frontend	Axios, React Router	API calls and page navigation
Backend	Spring Boot, Java 21	Server, business logic, REST APIs
Security	JWT, Spring Security, BCrypt	Login, roles, password hashing
Database	MySQL	Store users, orders, payments, etc.
PDF	OpenPDF	Generate invoice PDF

4. System Architecture (Simple Flow)

The application follows a simple three-layer flow:

- **Frontend (React)** → the user clicks something on a screen.
- **API call (Axios)** → the frontend sends a request to the backend.
- **Backend (Spring Boot)** → receives the request, runs the logic, and talks to the database.
- **Database (MySQL)** → stores or returns the data.
- **Response** → the backend sends the result back, and the frontend shows it.

In one line: **Frontend asks, backend answers, database remembers.**

Inside the backend, the request passes through layers: Controller (front door) → Service (brain / logic) → Repository (database access) → Entity (the database table).

5. User Roles

Role	What they can do
Customer	Browse menu, cart, coupons, pay, track order, invoice, cancel/refund
Admin	Manage pizzas, kitchen workflow, assign delivery, view analytics
Delivery	See assigned orders, update delivery status, collect cash (COD)

6. Main Features (Modules)

Login & Roles (JWT)

When a user logs in, the backend checks email and password. If correct, it returns a JWT token (like an entry pass). Every later request carries this token so the backend knows who the user is and what role they have. Only the right role can open the right pages (for example, only Admin can open Analytics).

Code location: Backend: controller/AuthController.java, config/SecurityConfig.java, security (JWT). Frontend: services/authService.js, context/AuthContext.jsx.

Menu & Cart

The customer sees all pizzas with price and veg/non-veg tags, adds items to the cart, and changes quantity. The cart is saved per user.

Code location: Backend: PizzaController.java, CartController.java, CartServiceImpl.java. Frontend: pages/Menu.jsx, pages/Cart.jsx, context/CartContext.jsx.

Coupons & GST Billing

On checkout, the customer can apply a coupon: PIZZA10 (10% off), FREEDEL (free delivery), or PARTY20 (20% off above Rs. 999). The bill clearly shows Subtotal + GST (5%) + Delivery Fee - Discount = Grand Total. The backend re-checks the coupon when the order is placed, so the price cannot be changed from the browser.

Code location: Backend: service/impl/CouponServiceImpl.java (validate), controller/CouponController.java. Frontend: pages/Checkout.jsx, services/couponService.js.

Place Order

When the customer places the order, the backend reads the cart, adds up the prices, applies GST, delivery fee and any coupon, saves the order as PLACED, and empties the cart.

Code location: Backend: service/impl/OrderServiceImpl.java (placeOrder). Frontend: pages/Checkout.jsx, services/orderService.js.

Payment (Online & Cash)

Two options: online (card/UPI) or Cash on Delivery (COD). For COD, the payment is marked success immediately and cooking begins. For online, it goes through a payment gateway layer that simulates an online payment (it is designed so a real gateway like Razorpay can be plugged in later with API keys).

Code location: Backend: service/impl/PaymentServiceImpl.java (createPayment), PaymentController.java. Frontend: pages/Payment.jsx, services/paymentService.js.

Pizza Recommendation

The system suggests pizzas to the customer using simple rule-based logic based on popularity and the customer's past orders. It does not use any external AI service like ChatGPT or Gemini; it is the project's own backend logic, which is fast, free, and fully explainable.

Code location: Backend: recommendation service in service/impl. Frontend: pages/Recommendations.jsx.

Order Tracking & ETA

The customer tracks the order through stages (Placed, Accepted, Preparing, Baked, Out for Delivery, Delivered) and sees an estimated time like 'Estimated delivery in 24 mins'. The page auto-refreshes.

Since there is no GPS, the ETA is an honest estimate based on order size, current stage and a fixed travel time.

Code location: Backend: service/impl/EtaServiceImpl.java (minutesRemaining), OrderMapper/DeliveryMapper. Frontend: components/customer/OrderTimeline.jsx, components/customer/EtaChip.jsx, pages/OrderDetails.jsx.

Invoice PDF

After payment, the customer can download a PDF invoice with the SmartPizza name, invoice number, items, quantities, prices, subtotal, GST, delivery fee, discount and grand total, using real order data. Only the order owner or an admin can download it.

Code location: Backend: service/impl/InvoiceServiceImpl.java (generateInvoicePdf), PaymentController.java (invoice). Frontend: pages/OrderDetails.jsx, services/paymentService.js (getInvoice).

Cancel Order & Refund

A customer can cancel only before the admin accepts the order (PLACED or CONFIRMED). After that it is blocked. If the order was paid online, the payment is safely marked REFUNDED inside the system and a refund message is shown. For COD there is nothing to refund.

Code location: Backend: service/impl/OrderServiceImpl.java (cancelOrder), OrderController.java. Frontend: pages/OrderDetails.jsx (handleConfirmCancel).

Admin Kitchen Workflow

The admin moves each order through the kitchen: Accept, Preparing, Baked, Ready for Pickup, on a live board that shows waiting time.

Code location: Backend: admin order controller, OrderServiceImpl. Frontend: pages/admin/Kitchen.jsx, components/admin/KitchenOrderCard.jsx, services/adminOrderService.js.

Assign Delivery & Route Optimization

The admin assigns the order to a delivery partner. The system recommends the best partner: the one with the fewest active deliveries right now, highlighted with a 'Recommended' badge. Since there is no GPS, this is real workload-based optimization.

Code location: Backend: service/impl/DeliveryOptimizationServiceImpl.java (recommend), DeliveryController.java. Frontend: pages/DeliveryDashboard.jsx (loadRec).

Delivery Workflow & COD Cash Collection

The delivery partner updates the order: Accept, Ready for Pickup, Out for Delivery, Delivered. For a cash order, they must press 'Collect Cash' before they can mark it Delivered, so cash orders cannot be closed without taking the money.

Code location: Backend: service/impl/DeliveryServiceImpl.java (collectCash and status updates). Frontend: pages/DeliveryDashboard.jsx, pages/delivery/Tracking.jsx, pages/delivery/Completed.jsx, services/deliveryService.js.

Admin Analytics

The analytics page shows real business data with charts: popular pizzas, busiest hours, menu mix, delivery efficiency (average delivery time, fastest partner, active workload) and coupon usage (coupons used, most used coupon, total discount given). All values come from real order and delivery data.

Code location: Frontend: pages/admin/Analytics.jsx, hooks/useAdminData.js (charts use Recharts).

7. Key API Endpoints

Method & Path	Purpose
POST /api/v1/auth/register	Register a new customer
POST /api/v1/auth/login	Login and get JWT token
GET /api/v1/pizzas	Get the pizza menu
POST /api/v1/orders	Place an order
PUT /api/v1/orders/{id}/cancel	Cancel an order (with refund if paid)
POST /api/v1/coupons/validate	Check coupon and get bill total
POST /api/v1/payments/create	Create a payment (online or COD)
GET /api/v1/payments/invoice/{orderId}	Download invoice PDF
GET /api/v1/admin/orders	Admin: list orders for kitchen
GET /api/v1/delivery/recommended-partner	Recommend best delivery partner
PUT /api/v1/delivery/{id}/collect-cash	Delivery: collect COD cash

8. Main Database Tables

Table	Stores
users	User accounts with role (Customer/Admin/Delivery)
pizzas / categories	Menu items and their categories
carts / cart_items	Each user's cart
orders / order_items	Orders, items, totals, coupon, discount, status
payments	Payment method, status, amount, refund
delivery_assignments	Which partner delivers which order, cash collected

9. How to Run the Project

Backend: Make sure MySQL is running. Open the backend folder and start the Spring Boot application. It runs on <http://localhost:8080>. On first start, it automatically creates an Admin and a Delivery account.

Frontend: Open the frontend folder, run **npm install**, then **npm run dev**. It runs on <http://localhost:5173>.

Default logins:

Role	Login Page	Email	Password
Admin	/admin/login	admin@smartpizza.ai	admin123
Delivery	/delivery/login	delivery@smartpizza.ai	delivery123

Customer	/register then /customer/login	(create your own)	(your own)
----------	--------------------------------	-------------------	------------

Sample coupons: PIZZA10, FREEDEL, PARTY20.

10. Honest Notes on Scope

- The recommendation, ETA and route features use clean rule-based logic, not heavy machine learning, because the project has no GPS data or large training dataset. The results are smart and fully explainable.
- The online payment uses a simulated gateway layer designed so a real provider can be added later with API keys. Cash on Delivery works fully end to end.
- Refunds are marked safely inside the system (status REFUNDED); no real bank/gateway refund is called.

11. Conclusion

SmartPizza AI is a complete, working full-stack application covering the full journey of an online food order: secure login, menu, cart, coupons and GST billing, online and cash payment, live tracking with ETA, PDF invoices, refunds, a kitchen workflow, smart delivery assignment, and a real analytics dashboard. Both the frontend and backend were designed and built as part of this capstone, with a clean, layered structure that is easy to maintain and extend.